
Desarrollo Web en Entorno Servidor - Segunda convocatoria

Desarrollo Web en Entorno Servidor - Segunda convocatoria

- **Ciclo Formativo:** Desarrollo de Aplicaciones Web (CFGS)
 - **Módulo:** Desarrollo Web en Entorno Servidor
 - **Duración:** 4 horas
 - **Curso:** 2025/2026
-

CONTEXTO

HabitatStudio es una plataforma web desarrollada en Laravel que pone en contacto a jóvenes en busca de alojamiento asequible con propietarios de casas compartidas. La plataforma nació como respuesta al problema de acceso a la vivienda que sufren los jóvenes en España, agravado por la actuación de fondos de inversión que adquieren inmuebles de forma masiva y elevan los precios del mercado de alquiler.

La aplicación ya dispone de un sistema de reservas (`bookings`) que relaciona casas (`casas`) con inquilinos (`users`). Tu tarea consiste en implementar el sistema de **testimonios y valoraciones** que actualmente aparece en la vista `home.blade.php` con datos estáticos, convirtiéndolo en una funcionalidad real respaldada por base de datos.

PREPARACIÓN DEL ENTORNO

- Haz un **fork** del repositorio **habitatstudio** y clónalo en tu equipo.
- Crea la rama **segundaConvocatoria**, al finalizar, tendrás que hacer un **Pull Request** de esa rama.

Notas:

- Copilot debe estar **deshabilitado** (**Disable Completions** ) y no se puede utilizar la inteligencia artificial para la resolución de ninguno de estos ejercicios.
 - Un ejercicio no será calificado como correcto si no son correctos los ejercicios anteriores.
 - Si prefieres desactivar los efectos del cursor, puedes poner a false la variable de entorno `CURSOR_EFFECTS`
 - Los esquemas **mermaid** los puedes encontrar en este mismo repositorio:
 - **esquema inicial**
 - **esquema final**
 - Sólo como consulta, puedes utilizar `php artisan test`. También tienes la posibilidad de ejecutar únicamente los tests relacionados con uno de los bloques. Las instrucciones de ejecución las encontrarás en el archivo `tests/README.md`.
-

RESULTADOS DE APRENDIZAJE EVALUADOS

Bloque	Resultado de aprendizaje	Calificación
A	RA5 — Separación lógica de negocio / presentación (Blade)	Nota independiente
B	RA6 — Acceso a almacenes de datos	Nota independiente
C	RA7 — Servicios web (API REST)	Nota independiente

Los bloques son **progresivos**: el Bloque B se apoya en las migraciones y modelos del Bloque A, y el Bloque C reutiliza la lógica de negocio implementada en el Bloque B. Se recomienda respetar este orden.

BLOQUE A — SEPARACIÓN LÓGICA DE NEGOCIO / PRESENTACIÓN

Resultado de aprendizaje 5

Objetivo: preparar el modelo de datos y las vistas Blade que sustituirán los testimonios estáticos de `home.blade.php` por datos reales procedentes de la base de datos.

A1 — Modificación de la tabla `casas` (1 pto.)

La tabla `casas` almacena actualmente el nombre del propietario como texto plano. Debes utilizar una nueva migración para modificar dicha tabla, **eliminando** la columna `nombre_propietario` sustituyéndola por una **clave ajena** `propietario_id` que reference a la tabla `users`.

No modifies la migración original de `users`. Crea una migración independiente del tipo `propietario_to_fk_in_casas_table`.

Actualiza también el modelo `Casa` para reflejar la nueva relación `propietario`, y el modelo `User` para añadir la relación inversa `casas`.

A2 — Atributos de perfil en `users` (1 pto.)

Crea una nueva migración que añada los siguientes campos a la tabla `users`:

- `avatar_url`: `varchar`, nullable. URL de la imagen de perfil del usuario.
- `descripcion_perfil`: `varchar`, nullable. Breve descripción que aparece bajo el nombre en los testimonios (p. ej. «**Estudiante de CIFP Carlos III**»).

No modifies la migración original de `users`. Crea una migración independiente del tipo `add_profile_fields_to_users_table`.

A3 – Migración y modelo Testimonio (1 pto.)

Crea la migración para la tabla `testimonios` con los siguientes atributos:

- `user_id`: clave ajena a `users`. Usuario que ha escrito el testimonio.
- `casa_id`: clave ajena a `casas`, **nullable**. Casa a la que hace referencia el testimonio. Será `NULL` cuando el testimonio sea sobre la plataforma en general, no sobre una casa concreta.
- `contenido`: `text`. Cuerpo del testimonio.
- `valoracion`: `decimal(3,1)`. Puntuación entre 0,0 y 5,0.
- `fecha_aprobacion`: `timestamp`, nullable. Fecha en que el propietario (o el administrador) aprueba el testimonio para que aparezca públicamente. Será `NULL` mientras el testimonio esté pendiente de aprobación.

La tabla incluirá también `id` y `timestamps`.

Crea el modelo `Testimonio` con:

- Las relaciones `User` (relación `usuario`) y con `Casa` (relación `casa`).
- Los métodos inversos en `User` (relación `testimonios`) y en `Casa` (relación `testimonios`).

A4 – Seeder (1 pto.)

Crea un `TestimonioSeeder` que inserte al menos **5 testimonios** de ejemplo con las siguientes características:

- Al menos 2 testimonios deben tener `fecha_aprobacion` rellena (testimonios aprobados y visibles).
- Al menos 1 testimonio debe tener `casa_id` a `NULL` (testimonio sobre la plataforma).
- Al menos 1 testimonio debe tener una valoración con decimal (p. ej. `4.5`).
- Los `user_id` deben corresponder a usuarios que sean inquilinos (que existan en `bookings`).

```
TestimonioSeeder se deberá ejecutar directamente al lanzar php artisan db:seed o con
migrate:fresh --seed
```

A5 – Vista de testimonios dinámica (2 ptos.)

Sustituye los testimonios estáticos de `home.blade.php` por datos reales. Para ello:

a) Modifica el método del controlador que carga `home.blade.php` para que recupere únicamente los testimonios con `fecha_aprobacion` no nula y los pase a la vista.

b) Modifica el **partial** `resources/views/partials/testimonios.blade.php` que reciba un array de objetos `Testimonio` y renderice los datos en tarjetas con:

- Avatar del usuario (`avatar_url`). Si es `NULL`, muestra un avatar por defecto.

- Nombre y `descripcion_perfil` del usuario.
- El `contenido` del testimonio.

lógica compleja: la lógica de cálculo de estrellas (cuántas enteras, si hay media y cuántas vacías).

A6 NO OBLIGATORIO - Estrellas de valoración en los testimonios (2 ptos.)

La valoración se representará con iconos de **Font Awesome**:

- Estrellas enteras: `fa-solid fa-star`
- Media estrella (cuando el decimal sea $\geq 0,5$): `fa-solid fa-star-half-stroke`
- Estrellas vacías: `fa-regular fa-star`

Deberás mostrar el número de estrellas (enteras, medias y vacías) que representen la valoración asociada al testimonio.

A7 NO OBLIGATORIO - Testimonios asociados a cada casa (2 ptos.)

En la vista `show` de una casa deben aparecer los testimonios asociados a esa casa y con `fecha_aprobacion` no nula.

BLOQUE B – ACCESO A ALMACENES DE DATOS

Resultado de aprendizaje 6

Objetivo: implementar el CRUD completo de testimonios con las reglas de negocio propias de la plataforma.

Este bloque requiere que la tabla `testimonios` y el modelo `Testimonio` del Bloque A estén operativos.

B1 – Controlador y rutas web (1 pto.)

Crea un `TestimonioController` con todas sus rutas bajo el prefijo `/testimonios`.

B2 – Listado y creación (1 pto.)

a) `index`: muestra todos los testimonios del usuario autenticado (aprobados y pendientes), ordenados por fecha de creación descendente. Puedes reutilizar el mismo contenido `blade` que el del `partial` `resources/views/partials/testimonios.blade.php`. Incluye en cada tarjeta el estado de aprobación (`Aprobado` / `Pendiente`).

b) `create` y `store`: formulario para que un inquilino cree un nuevo testimonio. El formulario incluirá:

- Un desplegable con las casas disponibles más la opción «Comentario general sobre la plataforma» (valor vacío que almacenará `NULL` en `casa_id`).
- Un campo de texto para el testimonio.

- Un campo numérico para la valoración (entre 0 y 5, con paso de 0,5).

Puedes utilizar el formulario existente en `resources/views/testimonios/create.blade.php`, añadiéndole algún elemento importante del que carece.

El método `store` debe:

- Asignar automáticamente el `user_id` del usuario autenticado, ignorando cualquier valor que pudiera venir en la petición.
- Poner `fecha_aprobacion` a `NULL`.
- Validar que el campo `contenido` no esté vacío, que `valoracion` esté entre 0 y 5, y que `casa_id`, si se envía, exista en la tabla `casas`.

B3 – Edición y eliminación (2 ptos.)

`edit` y `update`: permiten al usuario autenticado editar su propio testimonio, pero únicamente si aún no ha sido aprobado (`fecha_aprobacion` es `NULL`). Si el testimonio ya está aprobado, redirige al `index` con un mensaje de error.

`destroy`: Elimina el testimonio que coincide con el ID y que pertenece al usuario autenticado.

B4 – Restricción a inquilinos (2 ptos.)

Únicamente los usuarios que aparezcan como `inquilino_id` en al menos un registro de la tabla `bookings` podrán modificar (C - U - D) testimonios.

B5 – NO OBLIGATORIO Aprobación por el propietario (2 ptos.)

Implementa la ruta:

```
PUT /testimonios/{testimonio}/approve
```

que únicamente podrá ejecutar el **propietario de la casa** a la que hace referencia el testimonio. Al ejecutarse correctamente:

- Actualizará `fecha_aprobacion` con la fecha y hora actuales (`now()`).
- Redirigirá al listado de testimonios de esa casa con un mensaje de confirmación.

Si el testimonio tiene `casa_id` a `NULL` (testimonio general), la respuesta será `422` con un mensaje indicando que los testimonios generales no requieren aprobación.

Implementa la autorización mediante una `Policy` sobre el modelo `Testimonio`, con un método `approve` que devuelva `true` únicamente si el usuario autenticado es el propietario de la casa referenciada.

B6 - NO OBLIGATORIO - Revisión de testimonios (2 ptos.)

Implementa la ruta:

```
GET /testimonios/{testimonio}/revisar
```

que permitirá al administrador revisar los testimonios pendientes de aprobación. Al ejecutarse correctamente:

- Devolverá una vista con el contenido del testimonio y un formulario para aprobar o rechazar.
- Si se aprueba, actualizará `fecha_aprobacion` con la fecha y hora actuales (`now()`) y redirigirá al listado de testimonios con un mensaje de confirmación.
- Si se rechaza, eliminará el testimonio y redirigirá al listado de testimonios con un mensaje de confirmación.

Implementa la autorización mediante una `Policy` sobre el modelo `Testimonio`, con un método `before` que devuelva `true` únicamente si el usuario autenticado es el administrador.

Las credenciales del **administrador** serán las del único usuario que se genera al hacer el **seeder** de la base de datos.

BLOQUE C – SERVICIOS WEB

Resultado de aprendizaje 7

Objetivo: exponer la funcionalidad de testimonios como API REST, protegiendo los endpoints con autenticación Sanctum y aplicando las mismas reglas de negocio del Bloque B.

C1 – Listado público de testimonios aprobados (1 pto.)

Crea un `TestimonioResource` que exponga: `id`, `contenido`, `valoracion`, `fecha_aprobacion`, el `name` y `avatar_url` del usuario, y —si existe— el `nombre_casa` de la casa referenciada.

Implementa el endpoint público (sin autenticación):

```
GET /api/v1/testimonios
```

que devuelva una colección de `TestimonioResource` con todos los testimonios que tengan `fecha_aprobacion` no nula, ordenados por `fecha_aprobacion` descendente.

C2 – Testimonios de una casa (1 pto.)

Implementa el endpoint público:

```
GET /api/v1/casas/{casa}/testimonios
```

que devuelva los testimonios aprobados de una casa concreta. Si la casa no existe, la respuesta será `404`.

C3 – Creación de testimonio vía API (2 ptos.)

Implementa el endpoint autenticado (Sanctum):

```
POST /api/v1/testimonios
```

con las mismas reglas de validación y de negocio que `B2` y `B4`:

- `user_id` se asigna del usuario autenticado.
- `fecha_aprobacion` se asigna a `NULL`.
- Solo pueden crear testimonios los inquilinos; en caso contrario, `403`.
- Respuesta correcta: `TestimonioResource` con código `201`.

C4 – CRUD completo con Policy (3 ptos.)

Implementa los endpoints autenticados:

```
PUT /api/v1/testimonios/{testimonio}
DELETE /api/v1/testimonios/{testimonio}
PUT /api/v1/testimonios/{testimonio}/approve
```

Implementa la autorización mediante una `Policy` sobre el modelo `Testimonio`, con las siguientes reglas:

- `update`: solo el autor del testimonio y solo si no está aprobado. Devuelve el `TestimonioResource` actualizado.
- `destroy`: solo el autor. Devuelve `204 No Content`.
- `approve`: solo el propietario de la casa referenciada. Actualiza `fecha_aprobacion` y devuelve el `TestimonioResource` actualizado. Si `casa_id` es `NULL`, responde `422`.

Las respuestas de error de autorización deben devolver `403` en formato JSON, no una redirección. Asegúrate de que las rutas API utilizan el middleware `auth:sanctum` y de que la Policy se registra correctamente para el modelo `Testimonio`.

C5 NO OBLIGATORIO - Documentación con OpenAPI/Swagger (3 ptos.)

Documenta los endpoints anteriores utilizando OpenAPI/Swagger, asegurándote de incluir:

- Descripciones claras de cada endpoint, sus parámetros, respuestas y posibles errores.
- Esquemas de los recursos devueltos.

CRITERIOS DE CALIFICACIÓN POR RESULTADO DE APRENDIZAJE

Bloque A – RA5: Separación lógica de negocio / presentación

Ejercicio	Descripción	Puntuación
A1	Modificación de migración de <code>casas</code>	1 pto.
A2	Nuevos atributos de perfil en <code>users</code>	1 pto.
A3	Migración y modelo <code>Testimonio</code>	1 ptos.
A4	<code>TestimonioSeeder</code>	1 pto.
A5	Vista de testimonios dinámica con partial	2 ptos.
A6	Estrellas en la vista de testimonios	2 ptos.
A7	Testimonios asociados a casas	2 ptos.
Total RA5		10 ptos.

Bloque B – RA6: Acceso a almacenes de datos

Ejercicio	Descripción	Puntuación
B1	Controlador y rutas resource	1 pto.
B2	Listado, formulario de creación y <code>store</code>	1 pto.
B3	Edición con restricción y eliminación	2 ptos.
B4	Restricción a inquilinos	2 ptos.
B5	Aprobación por el propietario con Policy	2 ptos.
B6	Revisión de testimonios con Policy	2 ptos.
Total RA6		10 ptos.

Bloque C – RA7: Servicios web

Ejercicio	Descripción	Puntuación
C1	<code>TestimonioResource</code> y listado público	1 pto.
C2	Comentarios de una casa	1 pto.
C3	Creación autenticada con validaciones	2 ptos.
C4	CRUD completo con Policy en JSON	3 ptos.
C5	Documentación con OpenAPI/Swagger	3 ptos.
Total RA7		10 ptos.

COBERTURA DE CRITERIOS DE EVALUACIÓN

Criterio	Ejercicio(s) que lo cubren
RA5a – Ventajas de separar lógica y presentación	A5 (partial , lógica de estrellas en el modelo)
RA5b – Frameworks de separación	A1–A5 (uso de Laravel MVC, Blade)
RA5c – Objetos servidor para generar vista cliente	A3, A5b (controlador pasa datos a Blade)
RA5d – Formularios dinámicos	B2 (desplegable de casas generado dinámicamente)
RA5e – Configuración de la aplicación	<code>.env</code> y configuración del entorno (preparación)
RA5f – Mantenimiento de estado y separación	B3, B4 (sesión autenticada, lógica en modelo/policy)
RA5g – Principios y patrones OOP	A3 (relaciones Eloquent), B5 (Policy), C4 (Resource)
RA5h – Prueba y documentación	(reservado para tests automáticos en iteración futura)
RA6a – Tecnologías de acceso a datos	A1–A3 (migraciones, Eloquent ORM)
RA6b – Conexión con bases de datos	A1–A3 (migraciones ejecutadas sobre MySQL/SQLite)
RA6c – Recuperación de información	A5b, B1, B2 (consultas Eloquent con filtros y relaciones)
RA6d – Publicación de información recuperada	A5 (testimonios en <code>home.blade.php</code>)
RA6e – Conjuntos de datos	A4 (seeder con datos representativos)
RA6f – Actualización y eliminación	B3, B5 (update, delete, approve)
RA6g – Prueba y documentación	(reservado para tests automáticos en iteración futura)
RA7a – Características de los servicios web	C1–C4 (API REST sobre HTTP)
	C3, C4 (misma Policy reutilizada desde Bloque B)

Criterio	Ejercicio(s) que lo cubren
RA7b – Ventajas de servicios web para lógica de negocio	
RA7c – Tecnologías y protocolos	C1–C4 (JSON, HTTP, Sanctum)
RA7d – Estándares y arquitecturas	C1–C4 (REST, recursos anidados, códigos HTTP semánticos)
RA7e – Programar un servicio web	C1–C4
RA7f – Verificar el funcionamiento	C1–C4 (comprobable con Postman o similar)
RA7g – Consumir el servicio web	C1, C2 (endpoints públicos consumibles desde el navegador o Postman)
RA7h – Documentar un servicio web	C5 (documentación de endpoints con OpenAPI/Swagger)